

AGENTIC AI · MICROSOFT AUTOGEN

I built a team of **6 AI agents** that analyses any CSV from one plain-English question.

Upload a file. Ask a question. Get cleaned data, real statistics, charts and a written report — **with every number computed by code the agents wrote and ran themselves.**



Loader



Cleaner



Feature Eng.



Analyzer



Visualizer



Reporter



Code Executor

THE PROBLEM

Every data question means the same six chores.

1 · Load & profile the file

2 · Clean duplicates & gaps

3 · Engineer features

4 · Run the analysis

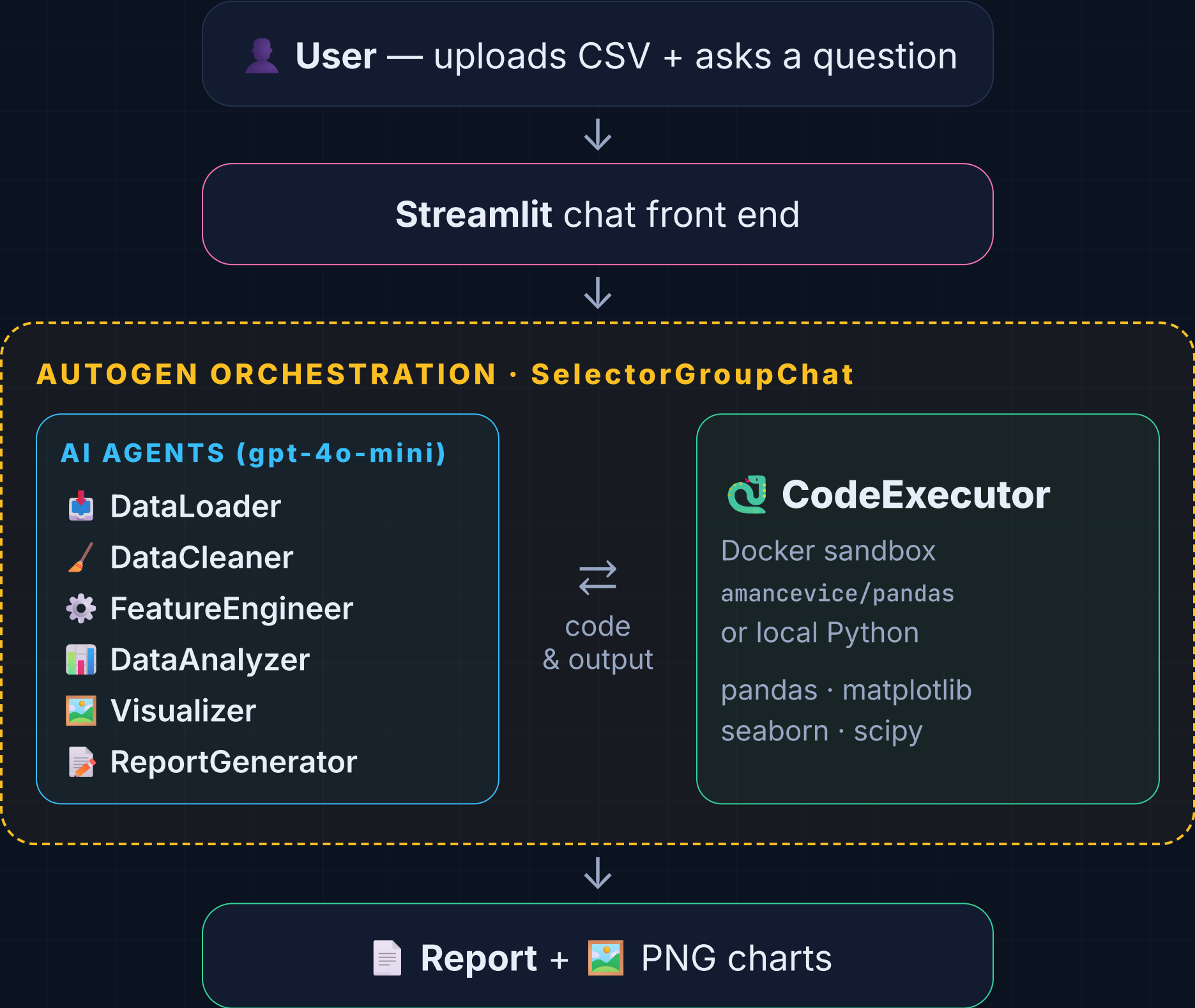
5 · Draw the charts

6 · Write it up

THE IDEA

Give each chore to a **specialist AI agent** — and make them prove every number by running **Python**.

How it fits together



MEET THE TEAM

One job each. No overlap.



DataLoader

Profiles the file: shape, types, missing values, duplicates



DataCleaner

Removes duplicates, fixes types, imputes gaps → `cleaned.csv`



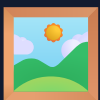
FeatureEngineer

Adds year / month / weekday, ratios, bins → `features.csv`



DataAnalyzer

Group-bys, trends, % change — answers every part of the question



Visualizer

Draws 2–4 labelled charts and saves them as PNG



ReportGenerator

Writes the Markdown report — only numbers the code produced

THE KEY DESIGN CHOICE

Agents don't guess numbers. They run code.

- 1 Agent writes a complete Python block
- 2  CodeExecutor runs it in the sandbox
- 3 Real output — or the traceback — goes back to the agent

Error?

Fix the code and retry (up to 5×)

Success?

Summarise and hand off to the next agent

A small **deterministic router** (`selector_func`) picks the next speaker.
No extra LLM call — so every run is **predictable, cheaper and easy to debug**.

STEP 1

Upload a CSV and pick a team

localhost:8501

Deploy ⋮

Data Analyzer GPT

Microsoft AutoGen agents analyse your CSV from a plain-English question.

OpenAI API key

.....

Model

gpt-4o-mini

Agent team

☒ Full pipeline (6 specialist agents)
☐ Quick (Analyzer + Code Executor)

Code execution

local

Docker isn't reachable, so agent-written code will run locally on this machine.

Upload a CSV file

sales.csv
21.9KB

+

Clear conversation

Ask questions about your data

▼ Preview: sales.csv (9 columns)

	order_id	order_date	region	category	units	unit_price	discount_pct	customer_rating	revenue
0	1381	2025-01-02	South	Furniture	6	345.27	5	5	1968.04
1	1104	2025-01-03	East	Grocery	3	14.83	15	4	37.82
2	1303	2025-01-04	West	Electronics	5	530.33	0	5	2651.65
3	1233	2025-01-08	East	Electronics	18	452.41	10	4	7329.04
4	1300	2025-01-08	East	Grocery	3	12.01	5	2	34.23
5	1170	2025-01-09	West	Clothing	18	49.48	15	5	757.04
6	1272	2025-01-09	South	Clothing	17	63.97	10	4	978.74
7	1369	2025-01-09	South	Grocery	10	17.87	20	4	142.96
8	1138	2025-01-12	East	Furniture	10	231.72	15	3	1969.62
9	1385	2025-01-12	West	Clothing	18	48.31	15	4	739.14

e.g. What are the monthly sales trends and which region performs best?

Full pipeline (6 agents) or Quick mode (1 analyst + executor). Demo data: **408 synthetic sales orders** with deliberate mess — duplicates, missing values, "N/A" in a number column.

STEP 2

Ask in plain English. Watch them work.

💬 "Which region has the highest total revenue, and how does revenue trend month by month?"

localhost:8501

Data Analyzer GPT

Microsoft AutoGen agents analyse your CSV from a plain-English question.

OpenAI API key

.....

Model

gpt-4o-mini

Agent team

Full pipeline (6 specialist agents)

Quick (Analyzer + Code Executor)

Code execution

local

Docker isn't reachable, so agent-written code will run locally on this machine.

Upload a CSV file

sales.csv21.9KB

+

Clear conversation

CodeExecutor

> Execution output

DataLoader

The dataset has 408 rows and 9 columns. The columns and their data types are as follows:

- order_id : int64
- order_date : str
- region : str
- category : str
- units : int64
- unit_price : float64
- discount_pct : int64
- customer_rating : float64
- revenue : float64

There are several missing values: 13 in unit_price , 22 in customer_rating , and 3 in revenue . Additionally, there are 8 duplicate rows in the dataset. A summary of the dataset indicates that the average revenue is approximately 2190.74 with a max revenue of 13379.15.

You can analyze which region has the highest total revenue and look into the month-to-month revenue trends next.

e.g. What are the monthly sales trends and which region performs best?

↑

Every message streams in live. Code runs, results come back, and the next agent picks up from saved files.

SELF-CORRECTING

It crashed. Then it fixed itself.

FeatureEngineer's first script failed — `order_date` was still text. It read the traceback, added one line, and re-ran. **No human involved.**

FeatureEngineer · attempt 2

FeatureEngineer

```
import pandas as pd

# Load the cleaned data
df = pd.read_csv('cleaned.csv')

# Ensure 'order_date' is treated as datetime
df['order_date'] = pd.to_datetime(df['order_date'])

# Feature Engineering
# Extract year, month, and day from 'order_date'
df['order_year'] = df['order_date'].dt.year
df['order_month'] = df['order_date'].dt.month
df['order_day'] = df['order_date'].dt.day
df['order_weekday'] = df['order_date'].dt.weekday # Monday=0, Sunday=6

# Create total revenue per region
total_revenue_by_region = df.groupby('region')['revenue'].sum().reset_index()
total_revenue_by_region.columns = ['region', 'total_revenue']

# Create a monthly revenue trend
df['order_month'] = df['order_date'].dt.to_period('M') # Monthly period
```

THE ANSWER

Real numbers, computed — not predicted

Top region

East

\$349,193

Peak month

Oct

\$114,329

Jan → Feb

+76.4%

month-on-month

DataAnalyzer · summary

DataAnalyzer

The analysis results are as follows:

1. Region with the Highest Total Revenue:

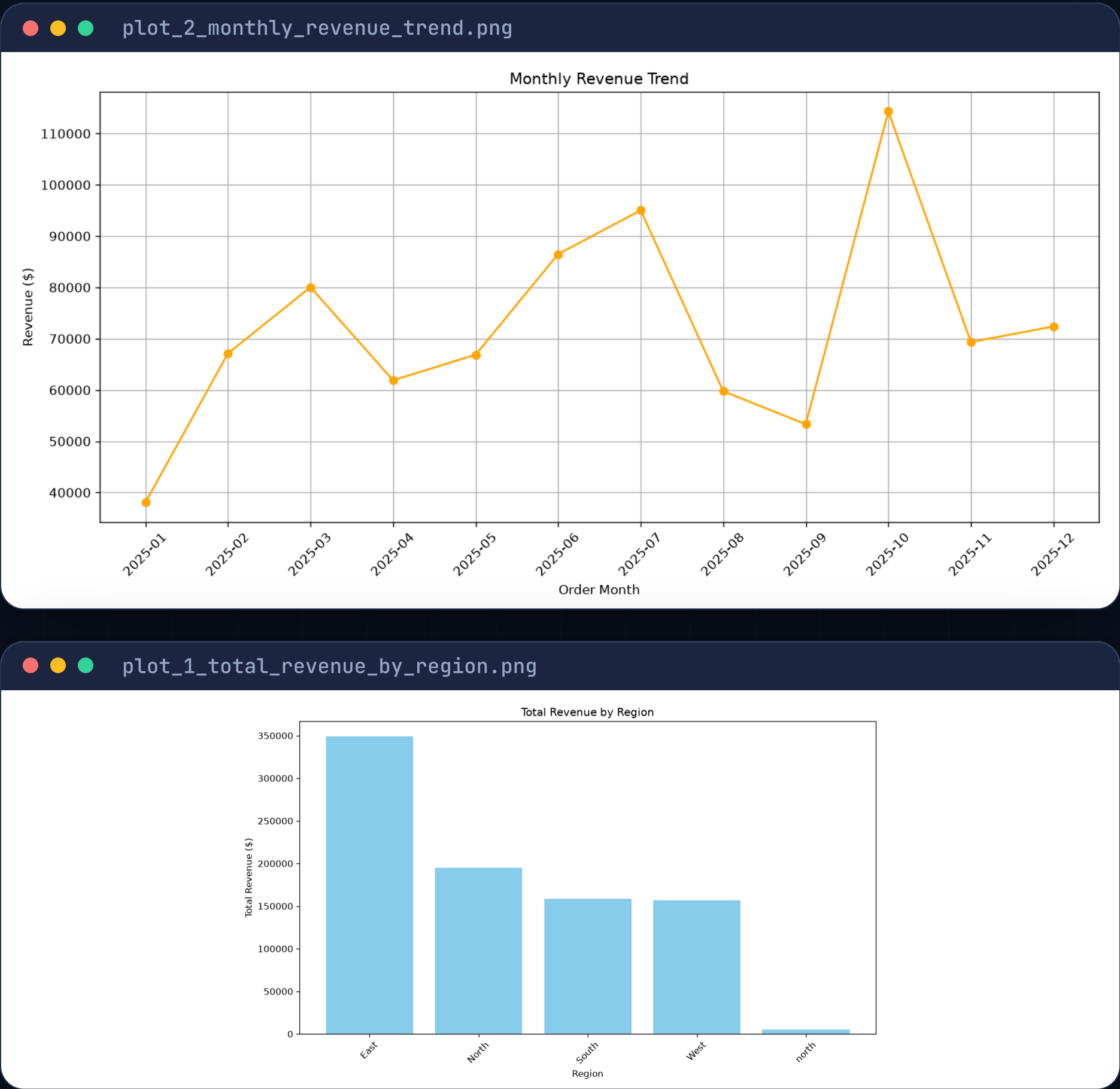
◦ Region: East

◦ Total Revenue: \$349,193.28

2. Monthly Revenue Trend:

Order Month	Revenue	Revenue Change (%)
2025-01	\$38,057.74	-
2025-02	\$67,138.17	76.41
2025-03	\$80,046.46	19.23
2025-04	\$61,890.45	-22.68
2025-05	\$66,874.90	8.05
2025-06	\$86,498.23	29.34
2025-07	\$95,036.69	9.87
2025-08	\$59,758.60	-37.12
2025-09	\$53,322.66	-10.77

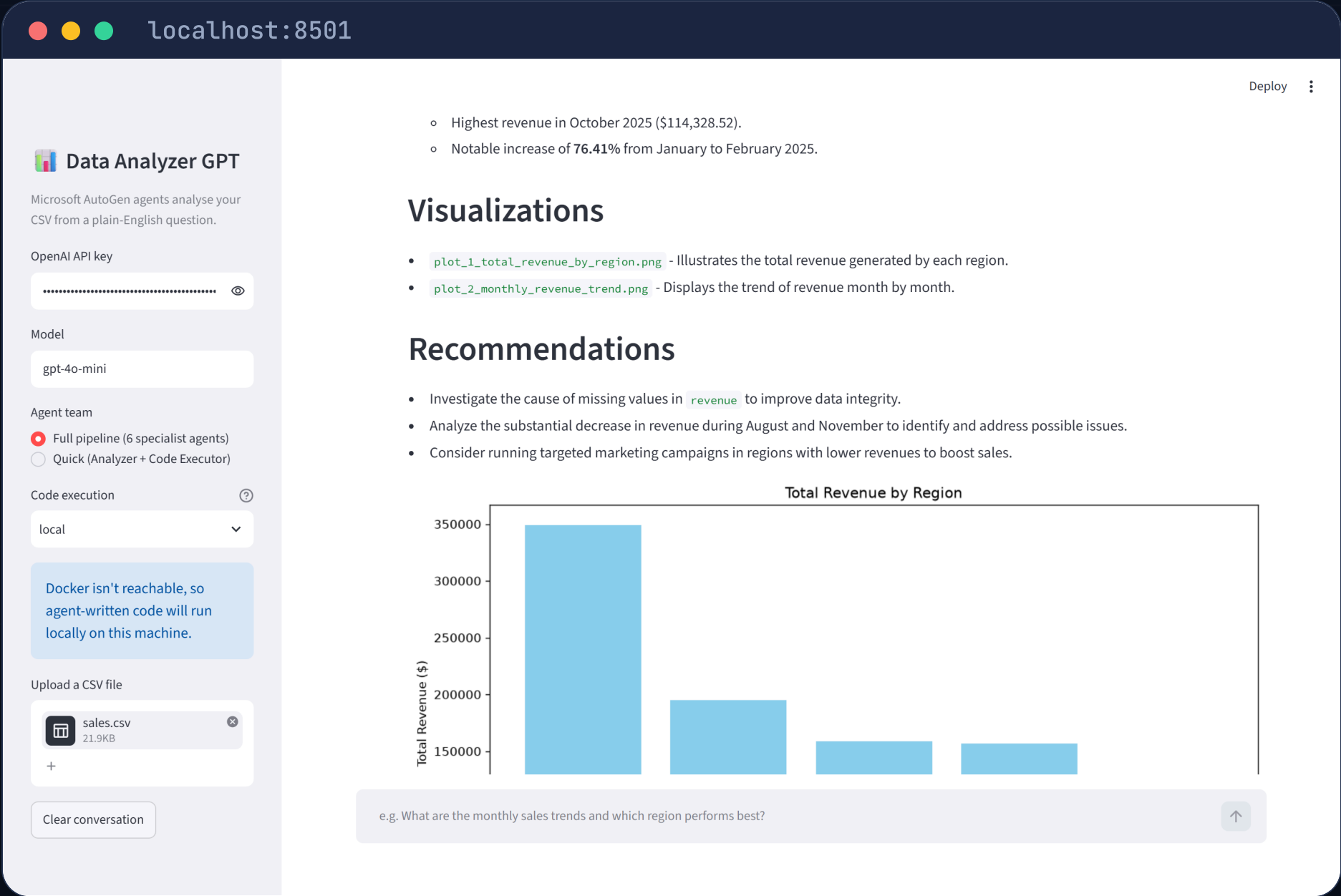
Charts it drew on its own



Spot the tiny **north** bar? The cleaner trimmed " north " but didn't fix its case — more on that in lesson 4.

STEP 3

A full report, ready to download



Answer

Dataset overview

Data cleaning

Key findings

Visualizations

Recommendations

One click exports it as Markdown.

4 lessons from real bugs

1 • Windows has no `sh`

An agent sent `pip install pandas` as a shell command and the whole run crashed. Fix: the executor reroutes `pip` to the app's Python — and hands errors **back to the agent** instead of crashing.

2 • Agents love to announce plans

"I will now engineer features..." — with no code — and the pipeline moved on. Fix: a stage **can't hand off until its code has actually run**.

3 • Use rules where rules work

Letting an LLM choose the next speaker adds cost and randomness. A 20-line router made the flow **deterministic and debuggable**.

4 • Still review the output

The cleaner left `north` and `North` as two regions. The answer held (East leads), but a human caught it in seconds. Agents speed you up — **they don't replace the review**.

BUILT WITH



Microsoft AutoGen 0.7



GPT-4o-mini



Python



pandas



matplotlib · seaborn



Streamlit



Docker sandbox

What question would **you** ask your data?

Tell me in the comments 📍 — I'll run the most interesting ones through the agents.



Like



Comment



Repost